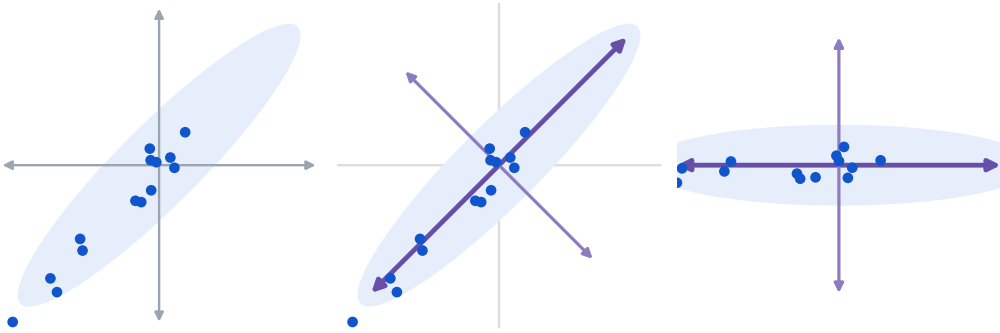


UNIT 02 · CHAPTER 6

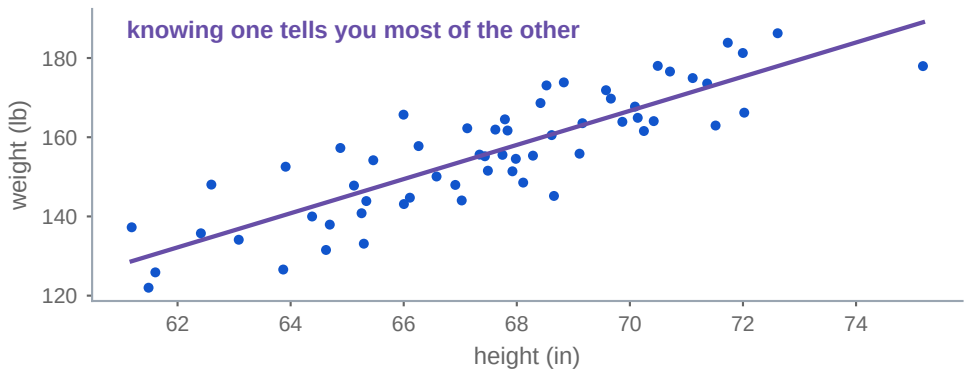
Principal Component Analysis

CSE 30124 · Introduction to Artificial Intelligence
William Theisen · University of Notre Dame

Too many features

Say we want to predict a student's GPA. We collect everything we can think of: **height, weight, hours of sleep, credit load, study time** and **assignment completion rate**.

Do you think all six features are equally useful? Some tell us almost nothing about GPA. Others **tell us** _____**:**

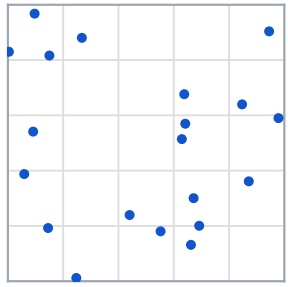


The curse of dimensionality

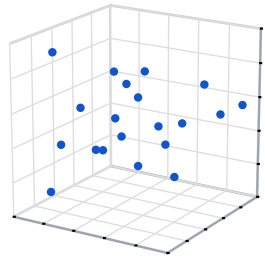
As we add more **dimensions** to our dataset, our data becomes more _____.



1 feature: 5 cells



2 features: 25 cells



3 features: 125 cells

The space grows _____, but our data doesn't.

This is the _____: in high dimensions every point looks far away from every other point

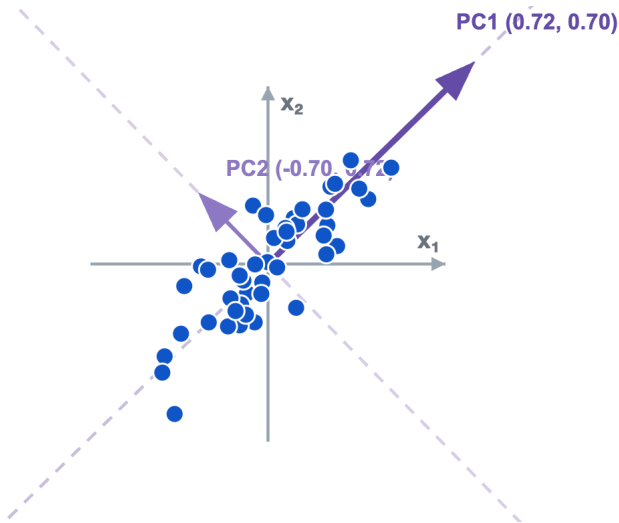
Quick check: we split every feature into 5 cells. How many cells are there with 6 features? With our 20 points, at most how many of them are occupied?

A new set of axes

Principal Component Analysis (PCA) is an _____ algorithm.

It finds us a _____ for our data, chosen so that:

- **Most of the variance** in the dataset lies along the first few axes.
- The axes are **orthogonal** to each other, so the new features are **uncorrelated**.



PCA is just a _____.

The road ahead

PCA is five steps.

1. **Standardize the data** — page 10
   Zero mean and unit variance for every feature.
2. **Covariance matrix** — page 12
   How much every pair of features moves together.
3. **Eigenvalues and eigenvectors** — pp. 13–16
   The new axes, and the variance along each.
4. **Sort them** — page 17
   Largest variance first.
5. **Choose how many to keep** — pp. 18–26
   Drop the axes that barely matter.

Notation

x_i	one example (a row); n examples, d features
μ, σ	a feature's average and standard deviation
C	the covariance matrix (d by d)
v	a direction (a row of length 1): a new axis
λ	the variance along that direction
$z = x\,v^T$	a point's position along the new axis

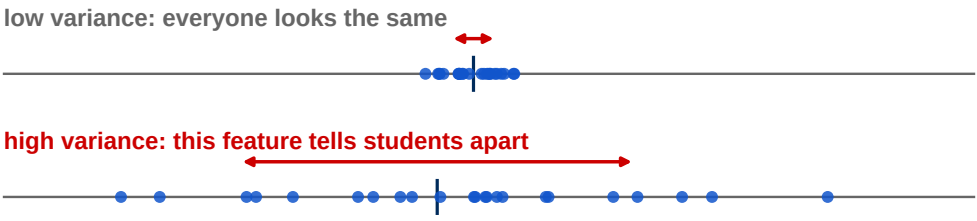
Variance

A dataset is only interesting if its samples are different from each other.

We measure how spread out a feature is with its _____.

[6.1] variance of one feature

$$\mathrm{Var}(x) = \frac{1}{n-1}\sum_i (x_i - \mu)^2$$



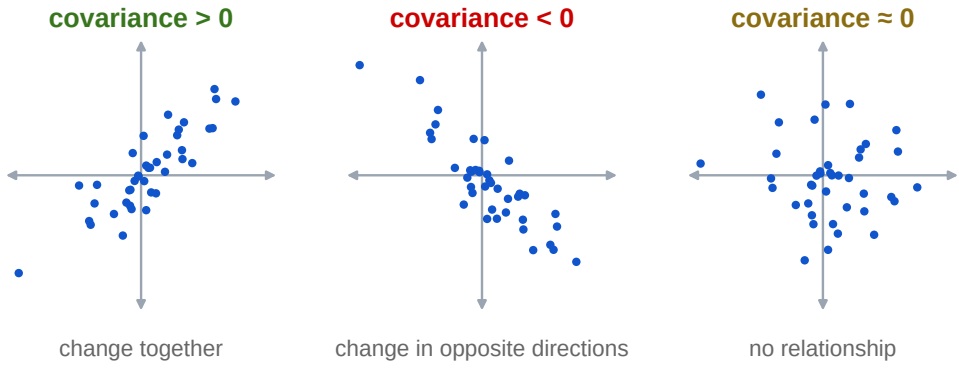
Typically, the **higher the variance** of a feature, the **more** _____ it can give us about the data.

Covariance

Features in our dataset often have some relationship *to each other*, like height and weight. We measure that with the _____ between two features a and b:

covariance of two features

$$[6.2] \quad \mathrm{Cov}(a, b) = \frac{1}{n-1} \sum_i (a_i - \mu_a)(b_i - \mu_b)$$



The more two features overlap (_____), the more overlapping information they carry.

Quick check: what is the covariance of a feature with itself?

The covariance matrix

We collect them all in one d by d table, the _____ C.

$$C = \begin{bmatrix} \mathrm{Var}(a) & \mathrm{Cov}(a,b) \\ \mathrm{Cov}(a,b) & \mathrm{Var}(b) \end{bmatrix}$$

- The **diagonal** holds each feature's variance.
- Everything **off the diagonal** is a covariance, and the matrix is symmetric.

[6.3] covariance matrix of centered data

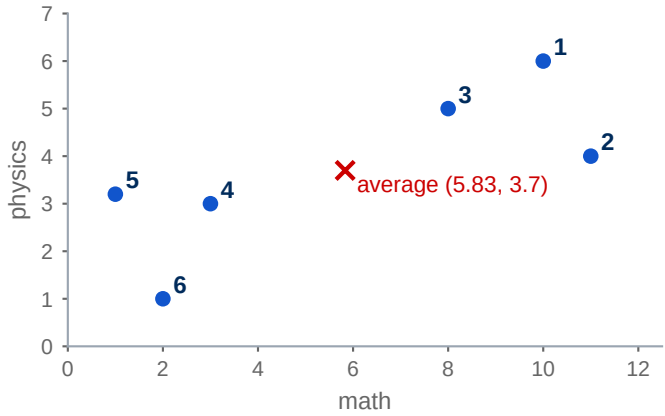
$$C = \frac{1}{n-1} X^T X$$

PCA's goal in one sentence: find new axes where this matrix has _____ everywhere off the diagonal.

Calculating PCA

Say we have the following **dataset** of six students' scores in Math and Physics:

Student	Math	Physics
1	10.0	6.0
2	11.0	4.0
3	8.0	5.0
4	3.0	3.0
5	1.0	3.2
6	2.0	1.0



Two features, but it looks like there's really **one story: "how strong is this student?"**

1. Standardize the data

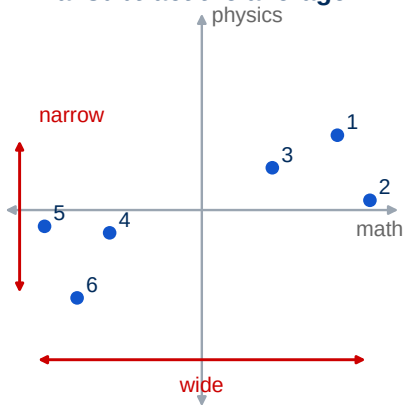
a. PCA first calculates the _____ of each feature and subtracts it, so the data is _____.

b. We then divide each feature by its _____, so both have a variance of _____.

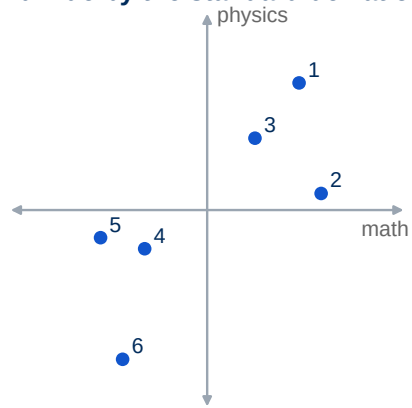
[6.4] standardize one value

$$x \leftarrow \frac{x - \mu}{\sigma}$$

a. subtract the average



b. divide by the standard deviation



Our standardized data

Student	Math	Physics	Product
1	0.96	1.32	1.27
2	1.19	0.17	0.20
3	0.50	0.75	0.37
4	−0.65	−0.40	0.26
5	−1.11	−0.29	0.32
6	−0.88	−1.55	1.37

Both columns now average _ with variance _.

Quick check: every product in the table is positive. What does that tell you about the covariance before we even compute it?

2. The covariance matrix

From the **covariance matrix** we can read the **relationships** between our features:

- **Covariance > 0:** features change together.
- **Covariance < 0:** features change opposite of each other.
- **Covariance ≈ 0:** features mostly don't have a relationship.

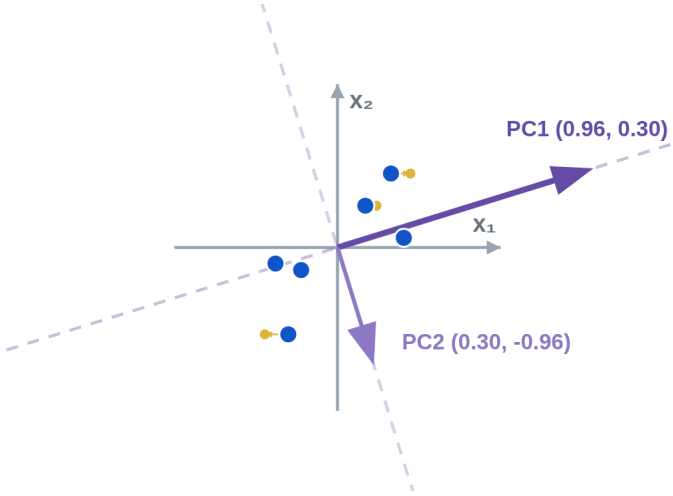
Ours is 0.76: _____, just like the picture.

What are we looking for?

A **direction** is a row v of length 1.

[6.6] position of x along direction v

$$z = x\,v^{\top}$$



The first new axis is the direction with the _____.

[6.7] variance along direction v

$$\mathrm{Var}(z) = v\,C\,v^{\top}$$

Why eigenvectors?

[6.8] the eigenvector equation

$$C v^T = \lambda v^T$$

Directions like that are called _____ of C, and their stretch factors λ are _____.

- The _____ are the directions our new axes point.
- The _____ are the variance along each of them.

3. Eigenvalues

A nonzero $\mathbf{v}$ can only be squashed to zero if the matrix $C - \lambda I$ has a _____.

[6.9] the characteristic equation

$$\det(C - \lambda I) = 0$$

↓ our C, with λ subtracted on the diagonal

$$\det\begin{bmatrix} 1-\lambda & 0.76 \\ 0.76 & 1-\lambda \end{bmatrix} = 0$$

↓ 2×2 determinant: $ad - bc$

$$(1-\lambda)^2 - 0.76^2 = 0$$

↓ square root of both sides

$$1 - \lambda = \pm 0.76$$

Two answers, one for each new axis:

3. Eigenvectors

Now we find the direction that goes with each eigenvalue.

$$\begin{bmatrix} -0.76 & 0.76 \\ 0.76 & -0.76 \end{bmatrix}\begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

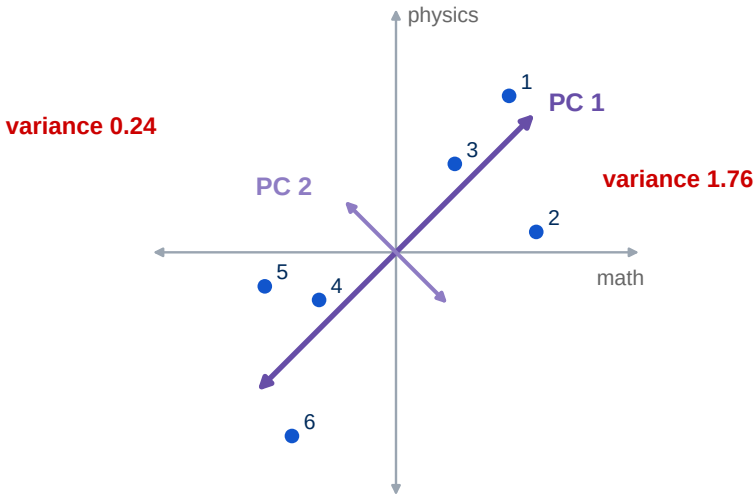
↓ both rows say the same thing

$$a = b$$

↓ scale to length 1

$$[6.11]\quad v_1 = \frac{1}{\sqrt{2}}[1,\ 1] \approx [0.71,\ 0.71]$$

Doing the same for $\lambda_2 = 0.24$ gives $a = -b$, so _____.



4. Sort them

We sort the eigenvalues from largest to smallest, and their eigenvectors along with them. The sorted directions are the _____:

	Direction	Variance λ	Share
PC 1	`[0.71, 0.71]`	1.76	**88%**
PC 2	`[-0.71, 0.71]`	0.24	12%

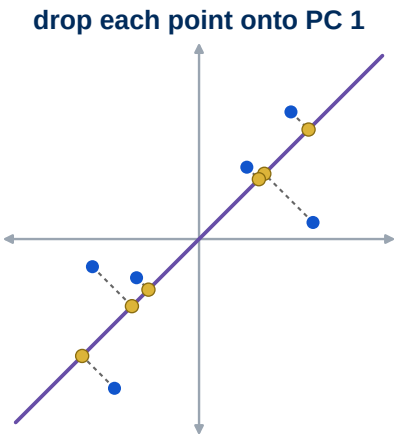
A component's share is its eigenvalue divided by the total. It's called the _____:

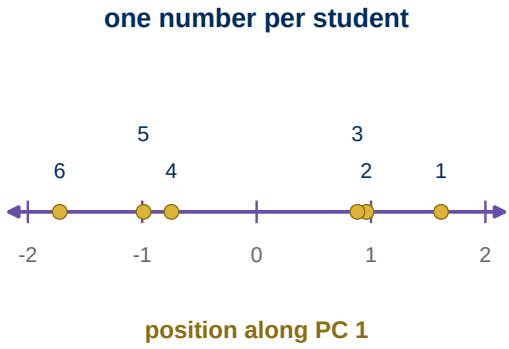
[6.12] share of the variance on component k

$$\frac{\lambda_k}{\lambda_1 + \lambda_2 + \cdots + \lambda_d}$$

5. Choose and project

PC 1 holds 88% of the variance, so let's keep only that one. Each student's new feature is their _____.





Student	1	2	3	4	5	6
Position on PC 1	1.61	0.96	0.88	−0.74	−0.99	−1.72

We went from two features to one, and **kept ____ of what made these students different.** That's _____.

Quick check: a new student is exactly average in math and one standard deviation above average in physics. Where do they land on PC 1?

Did it work?

Student	PC 1	PC 2	Product
1	1.61	0.26	0.42
2	0.96	−0.72	−0.69
3	0.88	0.18	0.16
4	−0.74	0.18	−0.13
5	−0.99	0.58	−0.57
6	−1.72	−0.48	0.82

This time the products cancel: they add up to ____.

The new features are _____.

Quick check: compare the Product column here with the one on page 11. What changed?

Going back

Almost: walk z steps along the direction we kept.

[6.14] rebuild a point from its position along v

$$\hat{x} = z\,v$$

- **What we lose** is the variance on the dropped components. Here that's $\lambda_2 = 0.24$, the missing _____.
- **Back to real units:** undo step 1 by multiplying by σ and adding μ. Student 1 comes back as about 10.8 in math and 5.7 in physics. They really scored 10 and 6.

$$\hat{x} = z_1 v_1 + z_2 v_2 + \cdots + z_k v_k$$

That won't scale

Finding the roots of a big polynomial is _____.

We instead use something called the _____ (SVD), which works on the data directly:

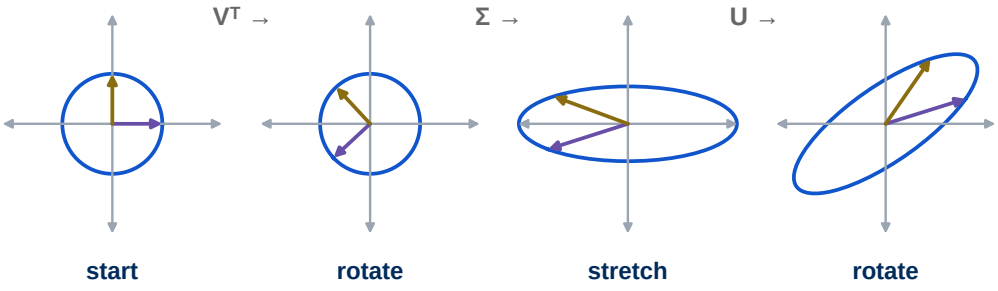
[6.15] singular value decomposition of any matrix

$$X = U \Sigma V^T$$

Remember, PCA is just trying to find a **new set of axes for the data**, and a change of axes is a _____.

Rotate, stretch, rotate

Any **linear transformation** can be described as a _____, then a _____ along the axes, then another _____.



- V^T **rotates.** Its rows are perpendicular directions of length 1.
- Σ **stretches.** It's zero except on its diagonal, which holds the **singular values** $\sigma_1 \geq \sigma_2 \geq \ldots$, already sorted.
- U **rotates** again.

The SVD hands us PCA

Because U is a rotation, $U^T U = I$ and it cancels:

$$C = \frac{1}{n-1} X^T X = \frac{1}{n-1} V \Sigma U^T U \Sigma V^T$$

$$\downarrow \; U^T U = I$$

$$[6.16] \quad C = V \frac{\Sigma^2}{n-1} V^T$$

- _____ are the **eigenvectors**: our principal components.
- _ holds the **eigenvalues**, after squaring and dividing by $n - 1$:

$$[6.17] \quad \lambda_k = \frac{\sigma_k^2}{n-1}$$

Steps 2, 3 and 4 all come out of one decomposition, **and we never build C at all.**

PCA in numpy

Here's our whole worked example. Standardize, then one call to the SVD:

```python
import numpy as np

# math and physics scores for six students
X = np.array([[10, 6], [11, 4], [8, 5],
              [3, 3], [1, 3.2], [2, 1]])

# step 1: standardize each column
Z = (X - X.mean(axis=0)) / X.std(axis=0, ddof=1)

# steps 2 to 4 in one call
U, S, Vt = np.linalg.svd(Z, full_matrices=False)
print(Vt.round(3))                      # directions
print(S.round(2))                       # singular values
print((S**2 / (len(X) - 1)).round(3))   # variances

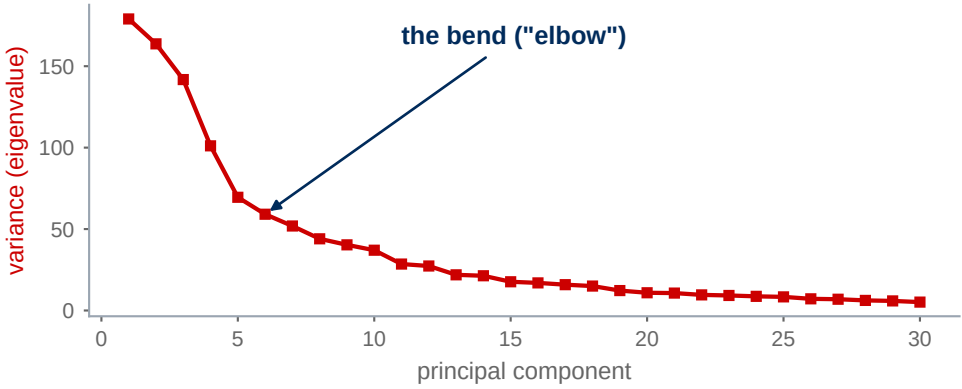
# step 5: keep only PC 1
z = Z @ Vt[0]
print(z.round(2))
```

OUTPUT

```
[[-0.707 -0.707]
 [-0.707  0.707]]
[2.97 1.1 ]
[1.758 0.242]
[-1.61 -0.96 -0.88  0.74  0.99  1.72]
```

Scree plots

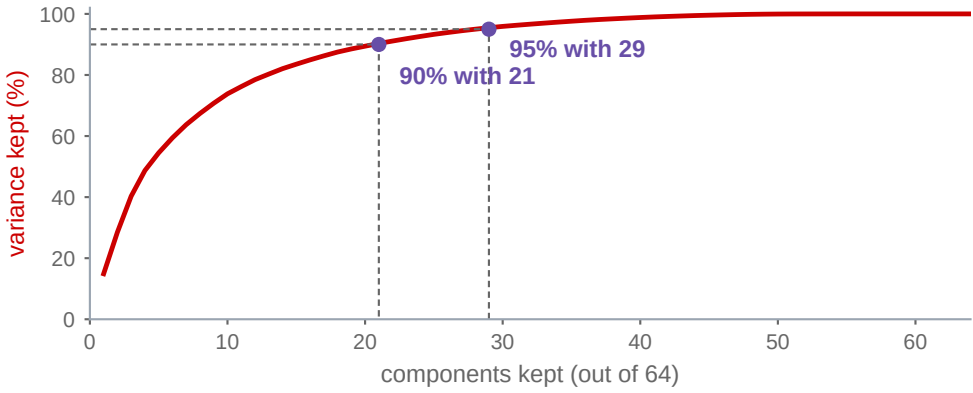
It's useful to plot the variance of each component, in order. This is called a _____:



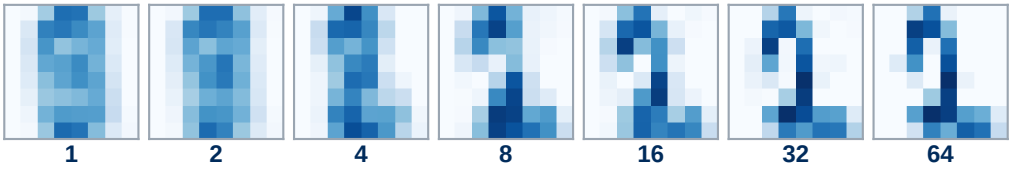
The bend is called the _____.

The new axes _____ **the variance**.

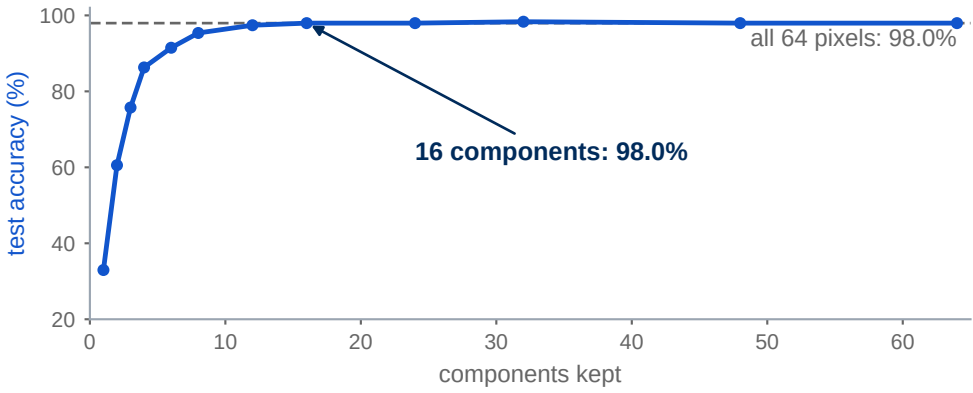
How many should we keep?



For our digits, __ **components keep 90% of the variance** and __ keep 95%.



Does the model mind?

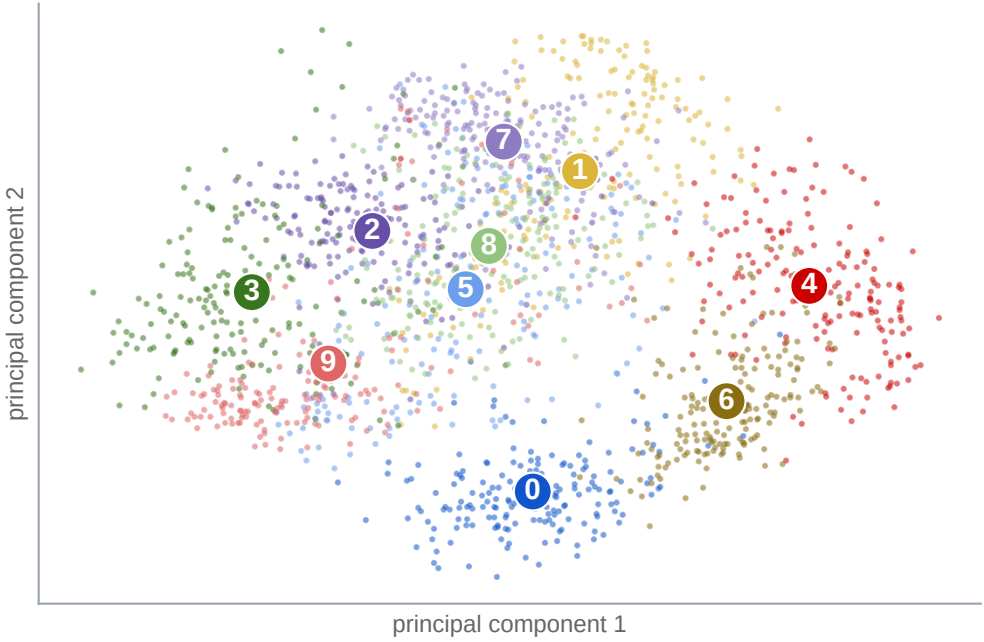


With just __ **of the 64 components** we match the accuracy of using every pixel. Fewer dimensions means:

- **Faster** training and prediction, and less to store.
- **Denser data**, so "nearby" means something again.
- **Less noise.** The components we drop are often mostly noise, so some models even improve.

Enabling visualizations

Another benefit of **PCA** is that it lets you _____.



PCA in Python

In practice we use scikit-learn's `PCA`, which centers the data and runs the SVD for us:

```
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = load_digits(return_X_y=True)   # 64 pixels per image
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

# learn the new axes from the training data only
pca = PCA(n_components=16).fit(X_tr)
Z_tr, Z_te = pca.transform(X_tr), pca.transform(X_te)
print(Z_tr.shape)
print(round(pca.explained_variance_ratio_.sum(), 3))

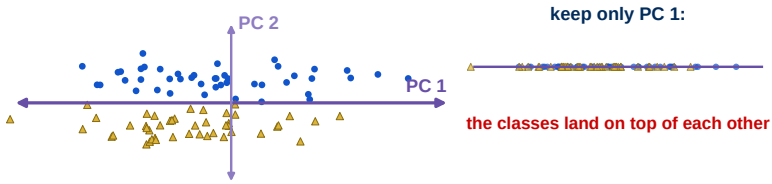
knn = KNeighborsClassifier(5).fit(Z_tr, y_tr)
print(round(knn.score(Z_te, y_te), 3))
```

OUTPUT

```
(1257, 16)
0.851
0.98
```

Where PCA misleads

PCA is unsupervised: it keeps the directions with the most variance, and **the most variance isn't always the _____.**



- **It's linear.** PCA can rotate, but it can't unbend. Curved structure needs something like the kernels of Chapter 4.
- **It's scale sensitive.** Measure a feature in different units and you get different components. Standardize first.
- **It's harder to explain.** "Component 3" is a mixture of every feature, not something you can point to.
- **It's swayed by outliers.** One extreme point can pull a whole component toward itself.
- **It can leak.** Fit PCA on the training data only, then apply those same axes to validation and test data.

Cheat sheet

The whole algorithm

1. **Standardize:** subtract each feature's average, divide by its standard deviation.
2. **Covariance matrix** C.
3. **Eigenvectors and eigenvalues** of C.
4. **Sort** by eigenvalue, largest first.
5. **Keep** the first k and project.

[6.3] covariance matrix

$$C = \frac{1}{n-1} X^T X$$

[6.8] directions and their variance

$$C v^T = \lambda v^T$$

[6.6] new feature, and going back

$$z = x v^T \qquad \hat{x} = z v$$

[6.12] explained variance ratio

$$\frac{\lambda_k}{\lambda_1 + \lambda_2 + \cdots + \lambda_d}$$

[6.17] from the SVD

$$X = U \Sigma V^T, \qquad \lambda_k = \frac{\sigma_k^2}{n-1}$$

Remember

- **Eigenvectors** are directions; **eigenvalues** are variance.
- The components are **orthogonal**, so the new features are uncorrelated.
- Rotating never changes the **total** variance.
- PCA is **unsupervised** and **linear**. Scale first; fit on training data only.

Practice problems

1. **Variance by hand.** A feature has the values 2, 4 and 6. What are its average, variance and standard deviation?
2. **Reading a matrix.** Two features have variances 4 and 9, and covariance −3. Write their covariance matrix. Do they move together or opposite?
3. **Two components.** Two standardized features have a covariance of 0.5. Find both eigenvalues and the share of the variance on PC 1.
4. **Projecting.** A standardized point is $\mathbf{x}$ = `[2, 1]` and PC 1 is $\mathbf{v}$ = `[0.6, 0.8]`. Is $\mathbf{v}$ a valid direction? Where does $\mathbf{x}$ land on it?
5. **How many?** A dataset's eigenvalues are 5, 3, 1.5 and 0.5. How much variance do the first two components keep? How many do we need for 95%?
6. **From the SVD.** A centered dataset with 10 points has singular values 6 and 2. What are the eigenvalues of its covariance matrix?
7. **Going back.** In problem 4 we kept only z. Rebuild the point from it. How far is the rebuilt point from the real $\mathbf{x}$ = `[2, 1]`?
8. **Units.** We run PCA on income (in dollars) and age (in years) without standardizing. What will PC 1 look like, and why?
9. **Reading a scree plot.** The eigenvalues are 4.0, 3.5, 0.3, 0.1 and 0.1. Where is the elbow? How many would you keep?
10. **Spot the leak.** A classmate fits PCA on the whole dataset, then splits it into training and test sets. What went wrong?